CODEALPHA

PYTHON PROGRAMMING INTERNSHIP

PROJECT REPORT

TASK 4: BASIC CHATBOT

A Simple Rule-Based Chatbot Using Python

Submitted By:

Pooja Subhasa Kenganingappanavar

Branch:

Computer Science Engineering

Internship Domain:

Python Programming

Organization:

CodeAlpha

Academic Year:

2026-2027

Project Type:

Internship Task – 4

BASIC SMART CHATBOT

DECLARATION

I hereby declare that the project entitled **“Basic Chatbot”** has been developed by me as part of the CodeAlpha Python Programming Internship – Task 4.

This project was developed using Python programming concepts and follows the requirements provided for the internship task. The application is a simple rule-based chatbot that accepts user input and provides predefined responses.

I declare that the work presented in this report is my own work carried out during the internship period for educational and learning purposes.

Name: Pooja Subhasa Kenganingappanavar

Branch: Computer Science Engineering

Internship: CodeAlpha Python Programming Internship

Date: 12-08-2026

Place: Bangaluru

Signature: psk

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to CodeAlpha for providing me with the opportunity to participate in the Python Programming Internship.

I am thankful for the practical tasks provided during the internship, which helped me apply Python programming concepts to develop useful applications.

I would also like to thank my teachers, mentors, friends, and family for their guidance, encouragement, and support throughout the development of this project.

The Basic Chatbot project helped me improve my understanding of Python functions, conditional statements, loops, input/output operations, and rule-based decision-making.

Finally, I completed this project and gained new knowledge and practical experience in developing a simple conversational application.

ABSTRACT

The **Basic Chatbot** is a simple rule-based conversational program developed using Python as part of the **CodeAlpha Python Programming Internship – Task 4.**

The main objective of this project is to create a chatbot that accepts input from the user and provides predefined responses based on the entered message.

The chatbot recognizes simple inputs such as **“hello”, “how are you”, “who are you”, “what is the time”**, and **“what is today’s date”.** Based on the user's input, the program provides suitable predefined responses.

The chatbot uses fundamental Python concepts including if-elif conditional statements, functions, loops, variables, strings, and input/output operations.

The submitted application also provides a graphical interface with quick-question buttons, a conversation area, a message input box, a Send button, and a Clear option.

This project demonstrates how basic programming logic can be used to create a simple interactive conversational application without requiring advanced artificial intelligence or machine learning.

TABLE OF CONTENTS

1. INTRODUCTION

A chatbot is a computer program designed to interact with users through conversations. Chatbots can receive messages from users and provide responses based on predefined rules or more advanced artificial intelligence techniques.

The Basic Chatbot developed in this project is a simple rule-based chatbot created using Python. It accepts text input from the user and compares the input with predefined conditions.

For example, when the user enters “hello”, the chatbot displays a greeting response. When the user asks “who are you”, it identifies itself as SmartBot. The interface also demonstrates time and date responses.

The project focuses on fundamental Python programming concepts and provides a simple introduction to conversational program development.

2. PROBLEM STATEMENT

The objective of this project is to develop a simple rule-based chatbot using Python.

The chatbot should accept input from the user and provide predefined responses according to the entered message.

The application should recognize basic messages and provide a relevant response. It should also provide a simple interface through which the user can send messages, use quick questions, and clear the conversation.

3. OBJECTIVES

• To develop a simple rule-based chatbot using Python.

• To accept text input from the user.

• To identify predefined user messages.

• To provide appropriate predefined responses.

• To use if-elif statements for decision-making.• To use functions for organizing chatbot logic.
• To use loops for continuous interaction.

• To demonstrate Python input and output operations.

• To provide a simple graphical chatbot interface.

• To improve programming and problem-solving skills.

4. SCOPE OF THE PROJECT

The scope of this project is to develop a basic rule-based chatbot that can interact with a user through text input and predefined quick questions.

The current version is intentionally simple. It does not depend on advanced artificial intelligence, machine learning, external APIs, or a database for its basic conversation logic.

The project provides a foundation for understanding how user input can be processed using programming logic and how a simple conversational application can be developed using Python.

5. TECHNOLOGIES AND TOOLS USED

5.1 Python

Python is the main programming language used to develop the Basic Chatbot.

5.2 Conditional Statements

if-elif statements can be used to compare the user's input with predefined messages and select an appropriate response.

5.3 Functions

Functions are used to organize the chatbot response logic and make the program easier to understand and maintain.

5.4 Loops

A loop can be used to allow the chatbot to continue interacting with the user until the conversation is ended.

5.5 Graphical User Interface

The submitted screenshots show a graphical interface containing a sidebar, quick questions, conversation area, input field, Send button, Clear button, and Settings option.

5.6 Visual Studio Code

Visual Studio Code can be used to write, edit, and execute the Python chatbot program.

6. SYSTEM REQUIREMENTS

Hardware: Computer or laptop, minimum 4 GB RAM, keyboard, basic processor, and sufficient storage space.

Software: Windows operating system, Python 3.x, and Visual Studio Code or another Python IDE.

7. PROJECT DESCRIPTION

The Basic Chatbot is a simple rule-based Python application that communicates with the user through a graphical chat interface and predefined responses.

The program accepts a message from the user and checks the message against a set of predefined conditions. The interface shown in the screenshots is branded as **SmartBot** and indicates that it is a rule-based assistant.

The application provides quick questions such as “Say Hello”, “How are you?”, “Who are you?”, “What can you do?”, “Tell me the time”, and “Today’s date”.

When the user enters “Hello”, the chatbot displays a greeting. When the user asks “Who are you”, the chatbot identifies itself as SmartBot. The screenshots also show time and date responses.

The chatbot follows predefined rules rather than generating answers using artificial intelligence. This makes the project simple and suitable for learning basic programming logic.

8. FEATURES OF THE CHATBOT

• User input through a message box.

• Quick-question options in the left sidebar.

• Greeting response for Hello.

• Identity response for “Who are you”.

• Time response.

• Date response.

• Clear chat functionality.

• Continuous conversation display.

• Simple dark-themed graphical interface.

• Online status indicator and Settings option.

9. METHODOLOGY

Step 1: Define Requirements – The CodeAlpha chatbot task requirements were analyzed.

Step 2: Define User Inputs – Basic messages and quick questions were selected.

Step 3: Define Responses – Predefined responses were created for recognized inputs.

Step 4: Create the Chatbot Function – A function was used to organize response logic.

Step 5: Accept User Input – The program accepts messages entered by the user.

Step 6: Compare Input – The entered message is compared with predefined conditions.

Step 7: Generate Response – The chatbot displays the corresponding response.

Step 8: Continue Conversation – The chat area keeps the interaction visible.

Step 9: Clear Conversation – The Clear option removes the current chat and starts again.

10. SYSTEM WORKFLOW

START → Display SmartBot interface → User selects quick question or enters message → Read user message → Check predefined rule → Generate matching response → Display response →Continue interaction → Clear chat or close application → END

11. ALGORITHM

1. Start the program.
2. Display the SmartBot welcome interface.
3. Wait for the user to select a quick question or enter a message.
4. Read the user's input.
5. Compare the input with predefined conditions.
6. If the input is a greeting, display the greeting response.
7. If the input asks about the bot, display the SmartBot identity response.
8. If the input asks for the time, display the current time.
9. If the input asks for the date, display the current date.
10. For another recognized message, display its predefined response.
11. Continue the conversation.
12. If Clear is selected, remove the displayed conversation and start again.
13. End the program when the user closes the application.

12. PYTHON CONCEPTS USED

12.1 If-Elif Statements: Used to compare user messages and determine the appropriate response.

12.2 Functions: Used to organize the chatbot's response logic.

12.3 Loops: Used to support repeated interaction.

12.4 Input: Used to receive messages from the user.

12.5 Output: Used to display chatbot responses.

12.6 Strings: Used to store and compare messages and responses.

12.7 Variables: Used to store user input and application data.

13. CONVERSATION LOGIC

User Input	Chatbot Response / Action
Hello	Displays a friendly greeting.
Who are you	Identifies itself as SmartBot and a rule-based chatbot.
What is the time	Displays the current time.
What is today’s date	Displays the current date.
Clear	Clears the current conversation and starts again.

Example Conversation

User: Hello

Chatbot: Hello! It's nice to meet you. How can I help you?

User: Who are you

Chatbot: My name is SmartBot. I'm a simple rule-based chatbot.

User: What is the time

Chatbot: The current time is displayed by the application.

User: What is today's date

Chatbot: Today's date is displayed by the application.

14. IMPLEMENTATION

The Basic Chatbot was implemented using Python's fundamental programming features and a graphical chat interface.

A function is used to organize the chatbot's response logic. The program accepts input from the user and checks the entered message using conditional rules.

When the user enters a recognized message, the corresponding predefined response is displayed in the conversation area. Quick-question buttons provide another convenient way to send predefined messages.

The application also demonstrates utility-style responses such as displaying the current time and date. The Clear button provides a simple way to reset the visible conversation.

The implementation follows the simplified requirements of the CodeAlpha Basic Chatbot task and demonstrates practical use of Python programming logic.

15. TESTING AND RESULTS

Test Case	User Action / Input	Expected Result	Result
1	Open application	SmartBot home interface appears	Pass
2	Enter Hello	Greeting response appears	Pass
3	Ask Who are you	SmartBot identity response appears	Pass
4	Ask What is the time	Current time is displayed	Pass
5	Ask What is today’s date	Current date is displayed	Pass
6	Click Clear	Conversation is cleared	Pass

The testing results shown above indicate that the main interface actions and predefined chatbot responses work as expected.

16. OUTPUT SCREENSHOTS

The following screenshots document the working output of the SmartBot application. Each figure shows a different interface state or chatbot interaction.

16.1 SmartBot Home Screen

This screenshot shows the main SmartBot interface. The screen contains the SmartBot title, online status, New Chat button, quick questions, Settings option, conversation area, Clear button, message input box, and Send button.

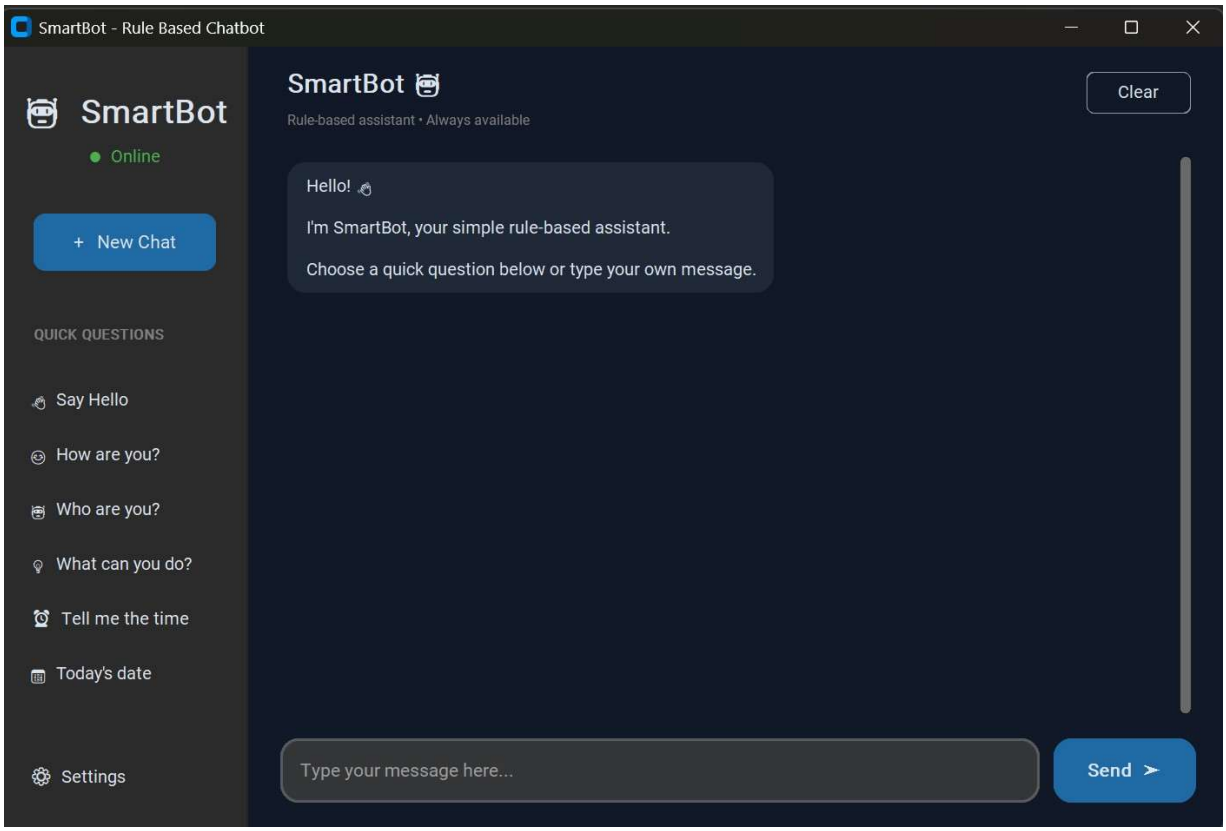

Figure 1: SmartBot Home Screen

16.2 General Greeting Chat

This screenshot demonstrates a basic greeting interaction. The user sends “Hello” and SmartBot displays a friendly predefined greeting response in the conversation area.

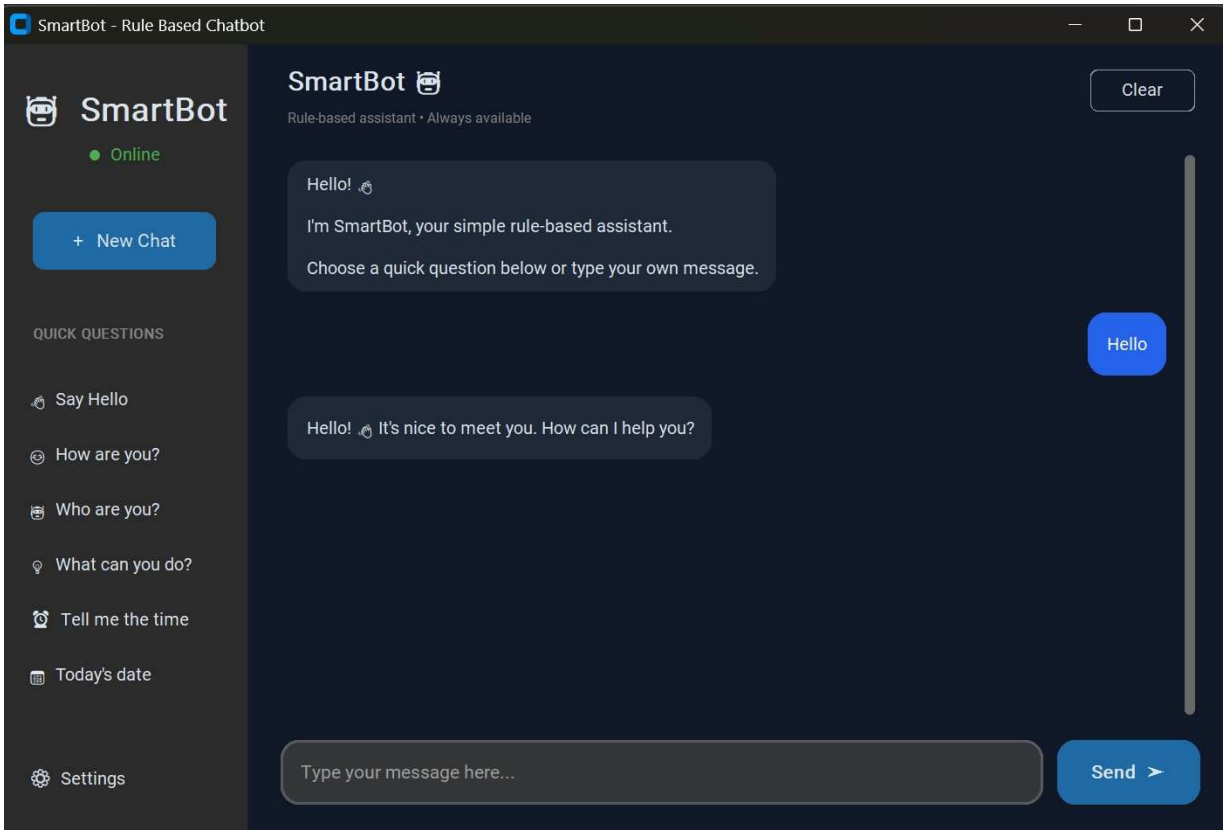

Figure 2: General Greeting Chat

16.3 Who Are You? Response

This screenshot demonstrates the chatbot identity response. After the user asks "Who are you", SmartBot identifies itself as a simple rule-based chatbot.

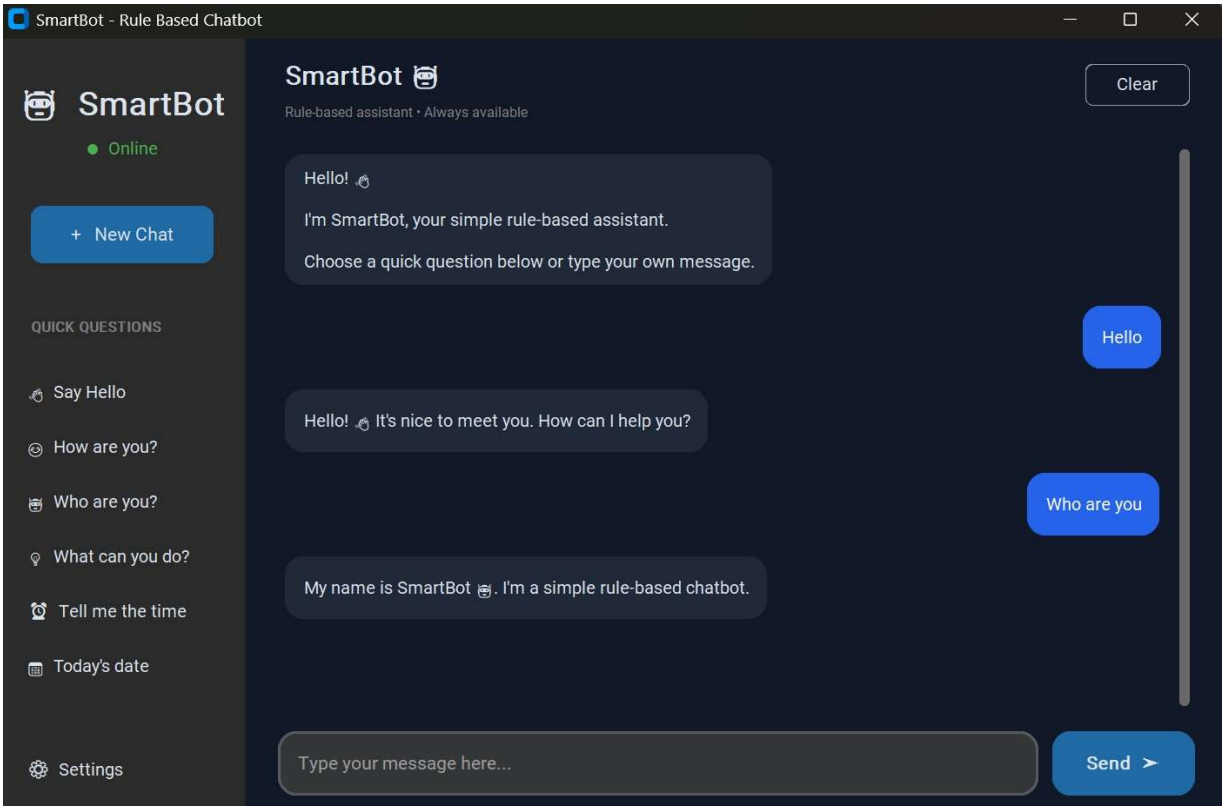

Figure 3: Who Are You? Response

16.4 Time Response

This screenshot demonstrates the time feature. The user asks “What is the time” and SmartBot displays the current time in the chat.

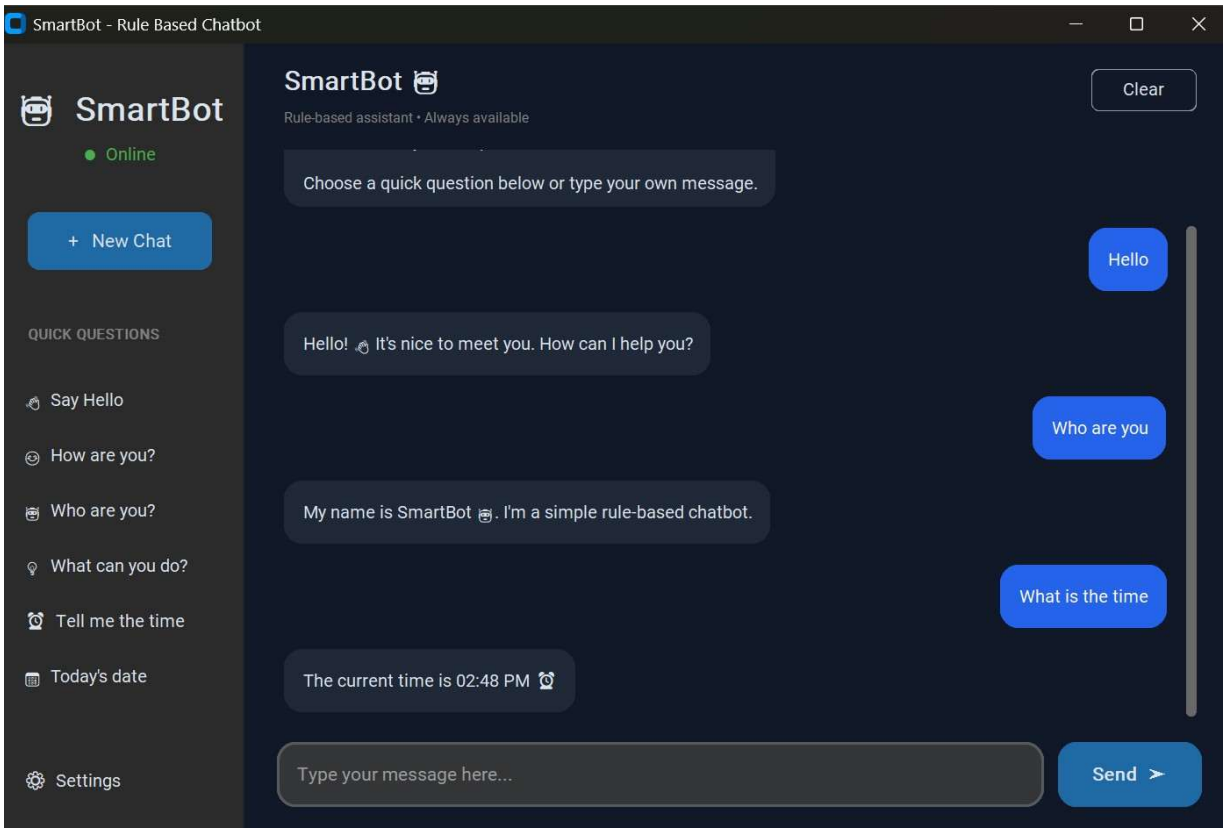

Figure 4: Time Response

16.5 Date Response

This screenshot demonstrates the date feature. The user asks “What is today’s date” and SmartBot displays the current date in the chat.

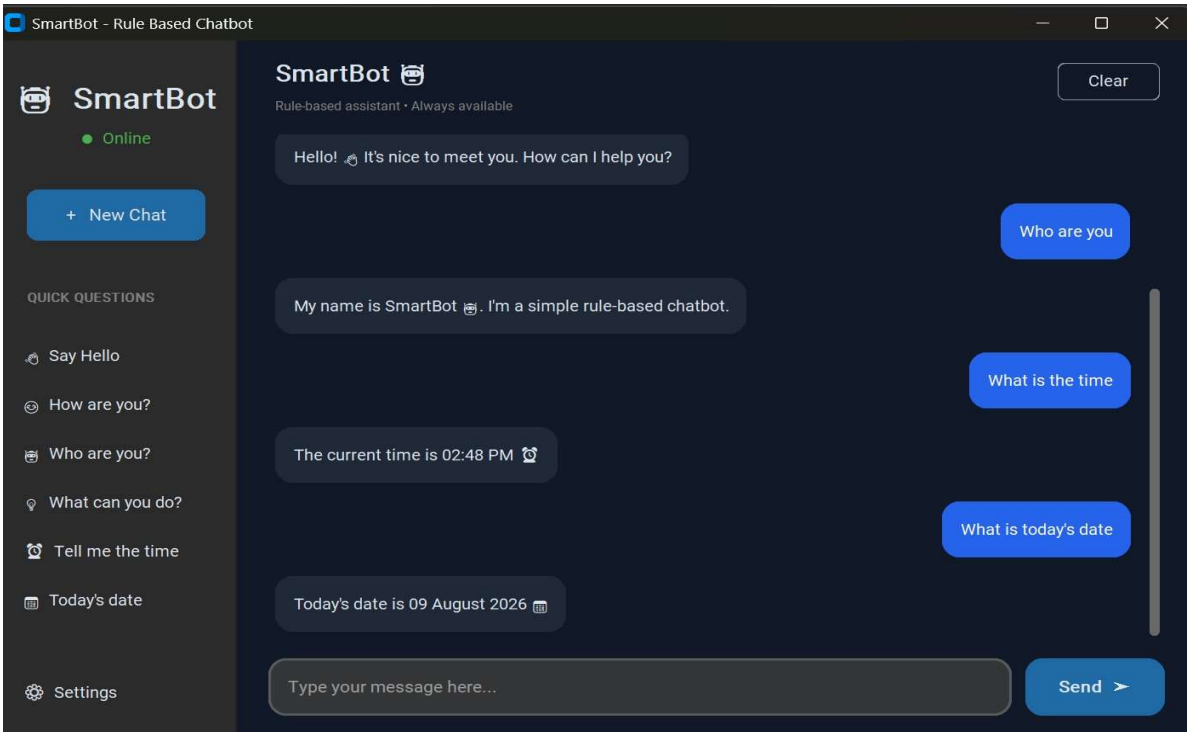

Figure 5: Date Response

16.6 Clear Chat Function

This screenshot demonstrates the Clear function. After the conversation is cleared, SmartBot displays a confirmation message and the chat area is reset for a new conversation.

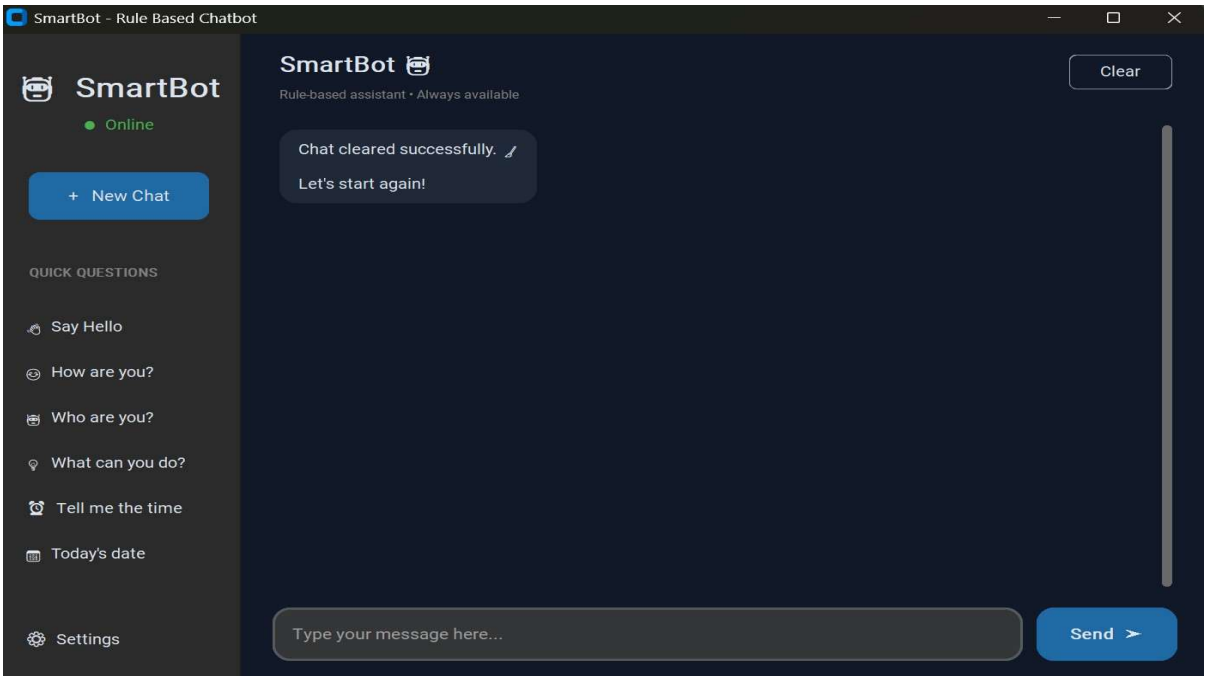

Figure **6: Clear Chat Function**

17. ADVANTAGES

• Simple and easy to understand.

• Easy to implement using Python.

• Demonstrates basic conversational logic.

• Provides immediate predefined responses.

• The basic rule-based logic does not require an internet connection.

• Does not require a database for the basic responses.

• Helps beginners understand conditional statements and functions.

• Provides practical experience with loops and user input.

• The interface is easy to use.

• Can be extended with additional responses and features.

18. LIMITATIONS

1. It supports only predefined inputs and rules.

2. It provides predefined responses rather than open-ended generated answers.

3. It has limited natural-language understanding.

4. The current project does not use advanced artificial intelligence.

5. It does not use machine learning.

6. It may not understand complex or unexpected questions.

7. It does not maintain long-term conversation context.

8. The number of available responses is limited by the implemented rules.

19. FUTURE ENHANCEMENTS

19.1 More User Inputs: Additional messages and questions can be added.

19.2 More Responses: The chatbot can be expanded with a larger collection of predefined responses.

19.3 Natural Language Processing: NLP techniques could be introduced to allow the chatbot to understand a wider range of user messages.

19.4 Artificial Intelligence: AI and machine learning techniques could be incorporated in an advanced version.

19.5 Voice Interaction: Voice input and output could be added in a future version.

19.6 Database Integration: A database could be used to store conversation information or chatbot responses.

20. CONCLUSION

The Basic Chatbot was successfully developed using Python as part of the CodeAlpha Python Programming Internship – Task 4.

The project successfully implements the main requirements of a simple rule-based chatbot by accepting user input and providing predefined responses. The submitted screenshots demonstrate greeting, identity, time, date, and clear-chat interactions.

The chatbot demonstrates the practical use of if-elif statements, functions, loops, strings, variables, and input/output operations. The graphical interface also makes the interaction simple and user-friendly.

Through this project, practical knowledge of rule-based decision-making and basic conversational program development was gained. Although the chatbot has a simple scope, it provides a strong foundation for understanding how conversational applications work and how more advanced chatbot features can be developed in the future.

21. REFERENCES

1. Python Documentation – Python Programming Language Documentation.

2. Python input() and print() documentation.

3. Python Functions and Conditional Statements documentation.

4. CodeAlpha Python Programming Internship Task Guidelines.

5. Visual Studio Code Documentation.

6. Python programming learning resources and tutorials used during project development.